



Module : Business Intelligence

Filière : 2ACI INF grp A

RAPPORT DE TP

Conception et Implémentation d'un Data Warehouse

Cas d'étude : Analyse décisionnelle d'une chaîne d'hôtels

Réalisé par :

Youssef FELLAH

Meryem EL FELLAH

Encadré par :

Pr. Abdelmajid BOUSSEL-
HAM

Année universitaire : 2025/2026

PARTIE I - Analyse Décisionnelle

1. Principaux besoins décisionnels de la direction

Le système opérationnel actuel se limite à la gestion transactionnelle, et la direction manque d'outils analytiques. Les besoins stratégiques identifiés sont les suivants :

- **Analyse Financière** : Analyser la rentabilité globale de la chaîne hôtelière.
- **Analyse Temporelle** : Identifier les périodes de forte activité pour adapter les ressources et les prix.
- **Analyse Géographique** : Évaluer les performances par région pour comparer les hôtels.
- **Analyse Marketing** : Mesurer l'efficacité des différents canaux de réservation.

2 & 3. Choix des KPI, formules et intérêts stratégiques

Voici 6 Indicateurs de Performance Clés (KPI) pertinents pour ce contexte :

1. Chiffre d'affaires (CA) total

- *Formule* : Somme de la mesure `montant_paiement` (ou `montant_total`).
- *Intérêt stratégique* : Jauger la santé financière macroscopique et mesurer la croissance globale.

2. Chiffre d'affaires par hôtel

- *Formule* : Somme de `montant_total` regroupée par `id_hotel`.
- *Intérêt stratégique* : Cibler précisément quels établissements sont les plus rentables ou nécessitent des ajustements.

3. Taux d'occupation

- *Formule* : $(\text{nombre total de nuits réservées} / (\text{capacité totale} \times \text{nombre de jours})) \times 100$.
- *Intérêt stratégique* : Mesurer l'utilisation des actifs physiques pour piloter le Yield Management (ajustement des prix selon la demande).

4. Durée moyenne de séjour

- *Formule* : Somme de `nombre_nuits` divisée par le nombre de réservations.
- *Intérêt stratégique* : Analyser le comportement client pour concevoir des offres promotionnelles allongeant les séjours (plus rentables logistiquement).

5. CA par canal de réservation

- *Formule* : Somme de `montant_total` regroupée par `id_canal`.
- *Intérêt stratégique* : Mesurer l'efficacité des leviers d'acquisition et optimiser les coûts de distribution (commissions des plateformes externes).

6. CA par type de chambre

- *Formule* : Somme de `montant_total` regroupée par `type_chambre`.
- *Intérêt stratégique* : Déterminer la rentabilité par gamme (standard, deluxe, suite) pour orienter les futurs investissements de rénovation.

PARTIE II - Conception du Modèle Dimensionnel

Table de faits et Dimensions

- **Table de faits principale** : Elle correspond aux réservations et se nomme `fact_reservation`.
- **Dimensions nécessaires** : Client, Hôtel, Chambre, Temps et Canal de réservation.

Hiérarchies analytiques

- **Hiérarchie temporelle** : Jour → Mois → Trimestre → Année.
- **Hiérarchie géographique** : Ville → Région → Pays.
- **Hiérarchie chambre** : Type → Catégorie.

Schéma en étoile

Le modèle s'articule autour d'une table centrale contenant les faits quantitatifs, directement reliée à cinq tables de dimensions descriptives.

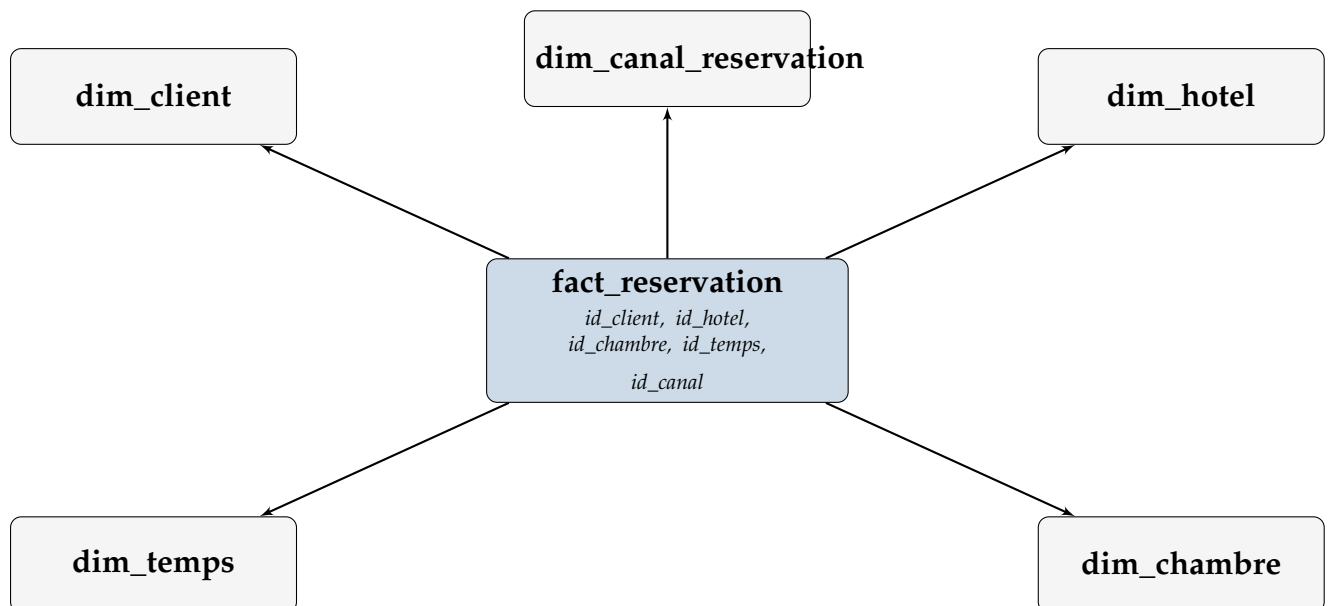


FIGURE 1 – Modèle en étoile du Data Warehouse

PARTIE III - Modélisation Logique

Structure de la table de faits : `fact_reservation`

Cette table concentre les métriques et les clés de relation :

- **Clés étrangères** : `id_client`, `id_hotel`, `id_chambre`, `id_temps`, `id_canal`.
- **Mesures** : `montant_total`, `nombre_nuits`, `montant_remise`, `montant_paiement`.

Structure des tables de dimensions

- **dim_client** : `id_client` (PK), `nom`, `ville`, `pays`.

- **dim_hotel**: id_hotel (PK), nom_hotel, ville, region, pays, capacite_totale.
- **dim_chambre**: id_chambre (PK), type_chambre, prix_nuit.
- **dim_temps**: id_temps (PK), date, jour, mois, trimestre, annee.
- **dim_canal_reservation**: id_canal (PK), nom_canal.

PARTIE IV - Implémentation PostgreSQL

Pour cette implémentation, le système de gestion de base de données PostgreSQL a été déployé de manière conteneurisée sur un environnement Fedora 43. Cette approche garantit l'isolation et la reproductibilité du Data Warehouse.

1. Préparation de l'environnement (Docker)

La première étape consiste à instancier un conteneur PostgreSQL fonctionnel.

Listing 1 – Lancement du conteneur PostgreSQL

```
docker run --name tp_datawarehouse \  
-e POSTGRES_USER=admin \  
-e POSTGRES_PASSWORD=admin \  
-e POSTGRES_DB=hotel_dw \  
-p 5432:5432 \  
-d postgres:16
```

Explication du fonctionnement : Cette commande télécharge l'image officielle `postgres:16` et lance un environnement isolé en arrière-plan (mode détaché `-d`). Elle initialise une nouvelle base de données nommée `hotel_dw` et crée un utilisateur super-administrateur via les variables d'environnement (`-e`). Le paramètre `-p 5432:5432` crée un pont réseau (Port Forwarding) permettant à la machine hôte Fedora de communiquer avec le port natif du conteneur.

Une fois le conteneur actif, la connexion au client interactif s'effectue via la commande suivante :

Listing 2 – Connexion à l'interface psql

```
docker exec -it tp_datawarehouse psql -U admin -d hotel_dw
```

Explication du fonctionnement : La directive `exec` exécute un processus supplémentaire dans le conteneur `tp_datawarehouse` déjà en cours d'exécution. Les arguments `-it` allouent un pseudo-terminal interactif, reliant les entrées clavier à l'outil client `psql`. Le client se connecte directement à la base `hotel_dw` avec les privilèges de l'utilisateur `admin`.

2. Création de la structure du Data Warehouse (DDL)

Le schéma en étoile nécessite la création des tables de dimensions avant la table de faits pour garantir l'intégrité référentielle.

Listing 3 – Script de création des tables

```
-- 1. Cr ation des dimensions
```

```
CREATE TABLE dim_client (  
    id_client SERIAL PRIMARY KEY,  
    nom VARCHAR(100) NOT NULL,  
    ville VARCHAR(100),  
    pays VARCHAR(100)  
);  
  
CREATE TABLE dim_hotel (  
    id_hotel SERIAL PRIMARY KEY,  
    nom_hotel VARCHAR(100) NOT NULL,  
    ville VARCHAR(100),  
    region VARCHAR(100),  
    pays VARCHAR(100),  
    capacite_totale INT NOT NULL  
);  
  
CREATE TABLE dim_chambre (  
    id_chambre SERIAL PRIMARY KEY,  
    type_chambre VARCHAR(50) NOT NULL,  
    prix_nuit DECIMAL(10, 2) NOT NULL  
);  
  
CREATE TABLE dim_temps (  
    id_temps INT PRIMARY KEY,  
    date_complete DATE NOT NULL,  
    jour INT NOT NULL,  
    mois INT NOT NULL,  
    trimestre INT NOT NULL,  
    annee INT NOT NULL  
);  
  
CREATE TABLE dim_canal_reservation (  
    id_canal SERIAL PRIMARY KEY,  
    nom_canal VARCHAR(100) NOT NULL  
);  
  
-- 2. Cr ation de la table de faits  
CREATE TABLE fact_reservation (  
    id_client INT REFERENCES dim_client(id_client),  
    id_hotel INT REFERENCES dim_hotel(id_hotel),  
    id_chambre INT REFERENCES dim_chambre(id_chambre),  
    id_temps INT REFERENCES dim_temps(id_temps),  
    id_canal INT REFERENCES dim_canal_reservation(id_canal),  
    montant_total DECIMAL(12, 2) NOT NULL,  
    nombre_nuits INT NOT NULL,  
    montant_remise DECIMAL(10, 2) DEFAULT 0,  
    montant_paiement DECIMAL(12, 2) NOT NULL,  
    PRIMARY KEY (id_client, id_hotel, id_chambre, id_temps, id_canal)  
);
```

Explication du fonctionnement : Le moteur SQL alloue l'espace de stockage physique pour chaque table selon un type strict. L'utilisation du pseudo-type SERIAL pour les dimensions g n re automatiquement une s quence auto-incr ment e pour les cl s primaires. La table fact_reservation centralise les mesures quantitatives (typ es en DECIMAL pour  viter les erreurs d'arrondi) et utilise la clause REFERENCES pour imposer des contraintes de cl s  trang res (Foreign Keys) liant les faits aux dimensions.

3. Insertion des données simulées (DML)

Un jeu de données réaliste est injecté dans le système pour permettre l'exécution des requêtes analytiques.

Listing 4 – Script d'insertion des données

```
-- Insertion dans les dimensions
INSERT INTO dim_canal_reservation (nom_canal) VALUES
('Site Web Direct'), ('Booking.com'), ('Expedia'), ('Agence locale'), ('
  T l phone'), ('App Mobile'), ('Airbnb'), ('Trivago'), ('Corporate'), ('
  Walk-in');

INSERT INTO dim_chambre (type_chambre, prix_nuit) VALUES
('Standard Simple', 50.00), ('Standard Double', 70.00), ('Deluxe Simple',
  100.00), ('Deluxe Double', 130.00), ('Suite Junior', 200.00), ('Suite
  Ex cutive', 350.00), ('Suite Prsidentielle', 800.00), ('Familiale',
  150.00), ('Standard Vue Mer', 90.00), ('Deluxe Vue Mer', 160.00);

INSERT INTO dim_hotel (nom_hotel, ville, region, pays, capacite_totale)
VALUES
('H tel Atlas', 'Casablanca', 'Casablanca-Settat', 'Maroc', 150), ('Riad
  Al-Andalus', 'Marrakech', 'Marrakech-Safi', 'Maroc', 40), ('Marina Bay
  Resort', 'Tanger', 'Tanger-T touan', 'Maroc', 200), ('Capital Inn', '
  Rabat', 'Rabat-Sal ', 'Maroc', 120), ('Desert Rose', 'Ouarzazate', '
  Dr a-Tafilalet', 'Maroc', 80), ('Ocean View', 'Agadir', 'Souss-Massa',
  'Maroc', 300), ('Fes Heritage', 'F s', 'F s-Mekn s', 'Maroc', 60), ('
  Atlas Mountains Lodge', 'Ifrane', 'F s-Mekn s', 'Maroc', 50), ('Casa
  Business Center', 'Casablanca', 'Casablanca-Settat', 'Maroc', 250), ('
  Marrakech Oasis', 'Marrakech', 'Marrakech-Safi', 'Maroc', 180);

INSERT INTO dim_client (nom, ville, pays) VALUES
('Ahmed Benali', 'Casablanca', 'Maroc'), ('Sophie Martin', 'Paris', 'France
  '), ('John Doe', 'New York', 'USA'), ('Fatima Zahra', 'Marrakech', '
  Maroc'), ('Hans M ller', 'Berlin', 'Allemagne'), ('Carlos Silva', '
  Madrid', 'Espagne'), ('Amine Sekkat', 'Rabat', 'Maroc'), ('Yuki Tanaka',
  'Tokyo', 'Japon'), ('Omar Berrada', 'Tanger', 'Maroc'), ('Emma Smith',
  'Londres', 'UK');

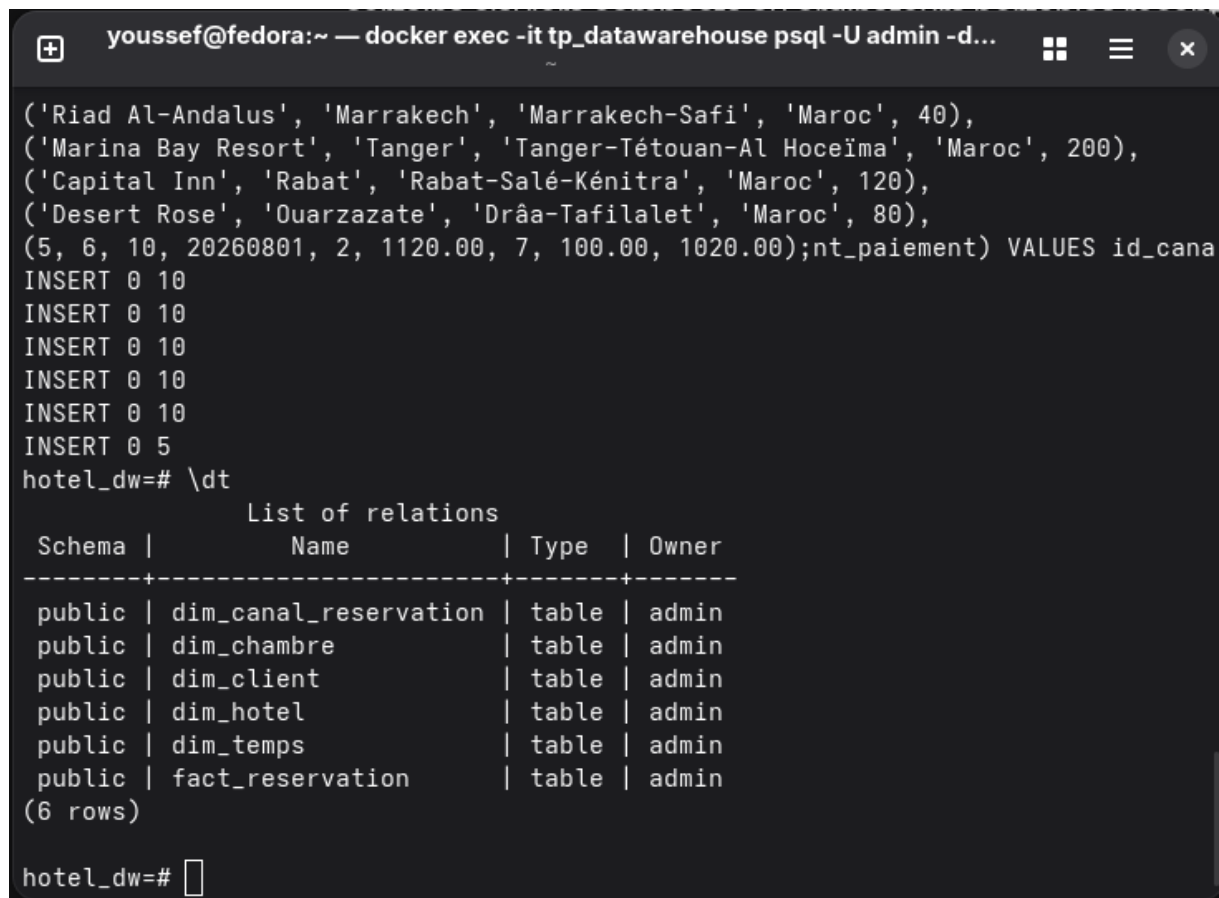
INSERT INTO dim_temps (id_temps, date_complete, jour, mois, trimestre,
  annee) VALUES
(20251201, '2025-12-01', 1, 12, 4, 2025), (20251215, '2025-12-15', 15, 12,
  4, 2025), (20251224, '2025-12-24', 24, 12, 4, 2025), (20251231, '
  2025-12-31', 31, 12, 4, 2025), (20260101, '2026-01-01', 1, 1, 1, 2026),
  (20260115, '2026-01-15', 15, 1, 1, 2026), (20260210, '2026-02-10', 10,
  2, 1, 2026), (20260214, '2026-02-14', 14, 2, 1, 2026), (20260615, '
  2026-06-15', 15, 6, 2, 2026), (20260801, '2026-08-01', 1, 8, 3, 2026);

-- Insertion dans la table de faits
INSERT INTO fact_reservation (id_client, id_hotel, id_chambre, id_temps,
  id_canal, montant_total, nombre_nuits, montant_remise, montant_paiement)
VALUES
(1, 1, 2, 20251201, 1, 210.00, 3, 0.00, 210.00),
(2, 2, 5, 20251224, 2, 800.00, 4, 50.00, 750.00),
(3, 3, 9, 20260101, 3, 450.00, 5, 0.00, 450.00),
(4, 1, 1, 20260214, 4, 100.00, 2, 10.00, 90.00),
(5, 6, 10, 20260801, 2, 1120.00, 7, 100.00, 1020.00);
```

Explication du fonctionnement : L'instruction `INSERT INTO` écrit les données directement dans les fichiers physiques gérés par PostgreSQL. L'utilisation d'une insertion en lot (Batch Insert), séparant plusieurs ensembles de valeurs par des virgules (), (), réduit les appels d'E/S (Entrées/Sorties) et optimise les performances. Les identifiants des tables de faits correspondent strictement aux clés générées dans les dimensions pour respecter les contraintes d'intégrité.

4. Visualisation et vérification

Afin de confirmer la bonne exécution des scripts, la méta-commande `\dt` permet de lister les relations créées, et une requête `SELECT` de base valide la présence des enregistrements.



```

('Riad Al-Andalus', 'Marrakech', 'Marrakech-Safi', 'Maroc', 40),
('Marina Bay Resort', 'Tanger', 'Tanger-Tétouan-Al Hoceïma', 'Maroc', 200),
('Capital Inn', 'Rabat', 'Rabat-Salé-Kénitra', 'Maroc', 120),
('Desert Rose', 'Ouarzazate', 'Drâa-Tafilalet', 'Maroc', 80),
(5, 6, 10, 20260801, 2, 1120.00, 7, 100.00, 1020.00);nt_paiement) VALUES id_cana
INSERT 0 10
INSERT 0 10
INSERT 0 10
INSERT 0 10
INSERT 0 10
INSERT 0 5
hotel_dw=# \dt
               List of relations
 Schema |          Name          | Type  | Owner
-----+-----+-----+-----
 public | dim_canal_reservation | table | admin
 public | dim_chambre           | table | admin
 public | dim_client            | table | admin
 public | dim_hotel              | table | admin
 public | dim_temps             | table | admin
 public | fact_reservation      | table | admin
(6 rows)

hotel_dw=#

```

FIGURE 2 – Vérification de la structure et du contenu des tables via `psql`

PARTIE V - Requêtes Analytiques

Cette section* présente l'implémentation et l'exécution des requêtes décisionnelles sous PostgreSQL, permettant d'extraire les indicateurs de performance clés (KPI) à partir du Data Warehouse.

12. Calculer le chiffre d'affaires par hôtel

```

SELECT
    h.nom_hotel,
    SUM(f.montant_paiement) AS chiffre_affaires
FROM
    fact_reservation f
JOIN
    dim_hotel h ON f.id_hotel = h.id_hotel
GROUP BY
    h.nom_hotel
ORDER BY
    chiffre_affaires DESC;

```

Mécanique de la requête : L'instruction JOIN fusionne la table de faits avec la dimension hôtel via la clé étrangère id_hotel. Le moteur SQL partitionne ensuite les enregistrements par établissement grâce à la clause GROUP BY. La fonction d'agrégation SUM() itère sur chaque sous-groupe pour additionner les paiements. Le tri décroissant (DESC) place l'hôtel le plus rentable en tête.

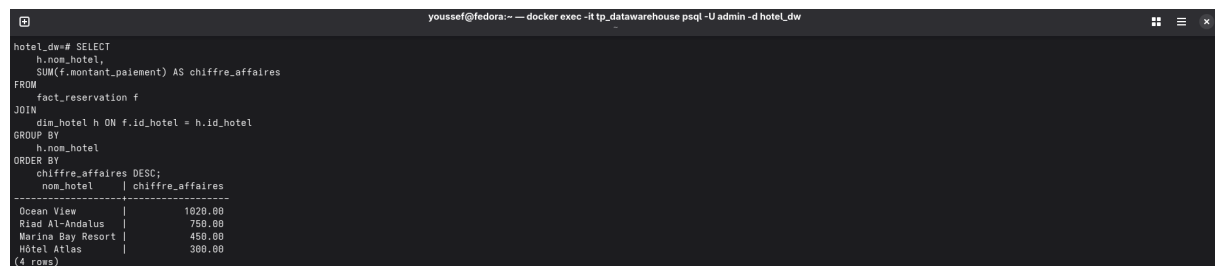


FIGURE 3 – Résultat : Chiffre d'affaires total par établissement hôtelier

13. Calculer le chiffre d'affaires par mois

```

SELECT
    t.annee,
    t.mois,
    SUM(f.montant_paiement) AS chiffre_affaires
FROM
    fact_reservation f
JOIN
    dim_temps t ON f.id_temps = t.id_temps
GROUP BY
    t.annee, t.mois
ORDER BY
    t.annee, t.mois;

```

Mécanique de la requête : Cette requête génère une série chronologique en croisant les faits avec dim_temps. Le regroupement s'effectue sur deux niveaux hiérarchiques : l'année, puis le mois. Déclarer t.annee en premier dans le GROUP BY est structurellement obligatoire pour éviter l'agrégation erronée de mois identiques appartenant à des années différentes.



```

hotel_dw=# SELECT
  t.annee,
  t.mois,
  SUM(f.montant_paiement) AS chiffre_affaires
FROM
  fact_reservation f
JOIN
  dim_temps t ON f.id_temps = t.id_temps
GROUP BY
  t.annee, t.mois
ORDER BY
  t.annee, t.mois;
annee | mois | chiffre_affaires
-----+-----+-----
2025  | 12   | 988.00
2026  | 1    | 450.00
2026  | 2    | 90.00
2026  | 8    | 1020.00
(4 rows)

```

FIGURE 4 – Résultat : Évolution chronologique du chiffre d'affaires

14. Calculer le taux d'occupation par hôtel

Note : La formule appliquée suppose une période d'analyse normalisée de 30 jours pour le jeu de données simulé.

```

SELECT
  h.nom_hotel,
  ROUND (
    (SUM(f.nombre_nuits)::numeric / (h.capacite_totale * 30)) * 100,
    2) AS taux_occupation_pct
FROM
  fact_reservation f
JOIN
  dim_hotel h ON f.id_hotel = h.id_hotel
GROUP BY
  h.nom_hotel, h.capacite_totale;

```

Mécanisme de la requête : Le calcul divise le volume de nuits vendues par l'offre maximale théorique (capacité × 30 jours). Le cast `::numeric` est strictement imposé au numérateur : sous PostgreSQL, la division de deux entiers tronque les décimales et renvoie 0. La fonction `ROUND ( )` intercepte le résultat pour le limiter à deux décimales.



```

hotel_dw=# SELECT
  h.nom_hotel,
  ROUND(
    (SUM(f.nombre_nuits)::numeric / (h.capacite_totale * 30)) * 100,
    2) AS taux_occupation_pct
FROM
  fact_reservation f
JOIN
  dim_hotel h ON f.id_hotel = h.id_hotel
GROUP BY
  h.nom_hotel, h.capacite_totale;
nom_hotel | taux_occupation_pct
-----+-----
Marina Bay Resort | 0.08
Ocean View | 0.08
Riad Al-Andalus | 0.33
Hotel Atlas | 0.11
(4 rows)

```

FIGURE 5 – Résultat : Taux d'occupation physique par hôtel sur 30 jours

15. Identifier le type de chambre le plus rentable

```

SELECT
  c.type_chambre,
  SUM(f.montant_paiement) AS rentabilite
FROM
  fact_reservation f
JOIN
  dim_chambre c ON f.id_chambre = c.id_chambre
GROUP BY
  c.type_chambre;

```

```

    c.type_chambre
ORDER BY
    rentabilite DESC
LIMIT 1;

```

Mécanique de la requête : Après avoir agrégé les montants financiers par gamme de chambre et trié les résultats du plus grand au plus petit, la clause propriétaire `LIMIT 1` intervient à la fin du pipeline d'exécution. Elle force le planificateur à tronquer l'ensemble du jeu de résultats pour ne renvoyer que le tuple de tête au client.

```

hotel_dw=# SELECT
  c.type_chambre,
  SUM(f.montant_paiement) AS rentabilite
FROM
  fact_reservation f
JOIN
  dim_chambre c ON f.id_chambre = c.id_chambre
GROUP BY
  c.type_chambre
ORDER BY
  rentabilite DESC
LIMIT 1;
 type_chambre | rentabilite
-----
Deluxe Vue Mer |      1020.00
(1 row)

```

FIGURE 6 – Résultat : Catégorie de chambre générant le revenu maximal

16. Calculer le chiffre d'affaires par canal de réservation

```

SELECT
    cr.nom_canal,
    SUM(f.montant_paiement) AS chiffre_affaires
FROM
    fact_reservation f
JOIN
    dim_canal_reservation cr ON f.id_canal = cr.id_canal
GROUP BY
    cr.nom_canal
ORDER BY
    chiffre_affaires DESC;

```

Mécanique de la requête : L'opérateur d'agrégation `SUM` s'applique de manière partitionnée sur la dimension des canaux d'acquisition. Le moteur de base de données alloue un accumulateur par valeur distincte trouvée dans la colonne `nom_canal` et additionne séquentiellement les faits financiers correspondants.

```

hotel_dw=# SELECT
  cr.nom_canal,
  SUM(f.montant_paiement) AS chiffre_affaires
FROM
  fact_reservation f
JOIN
  dim_canal_reservation cr ON f.id_canal = cr.id_canal
GROUP BY
  cr.nom_canal
ORDER BY
  chiffre_affaires DESC;
 nom_canal | chiffre_affaires
-----
Booking.com |      1770.00
Expedia    |       450.00
Site Web Direct |     210.00
Agence de voyage locale |      90.00
(4 rows)

```

FIGURE 7 – Résultat : Performances financières par vecteur d'acquisition

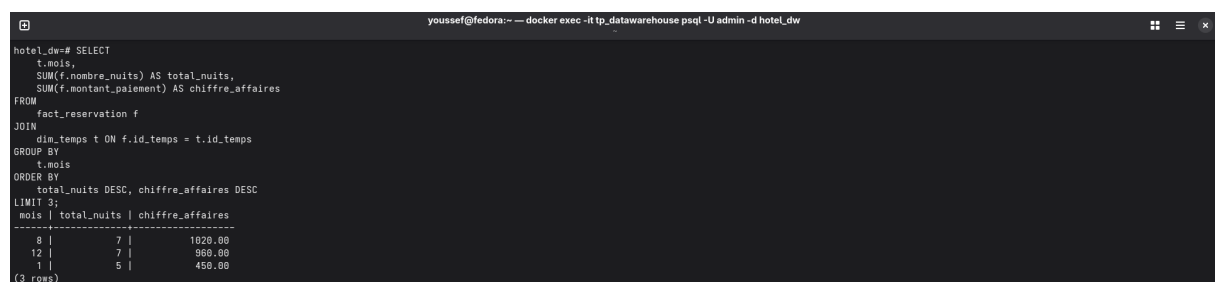
17. Identifier les mois de haute saison

```

SELECT
    t.mois,
    SUM(f.nombre_nuits) AS total_nuits,
    SUM(f.montant_paieement) AS chiffre_affaires
FROM
    fact_reservation f
JOIN
    dim_temps t ON f.id_temps = t.id_temps
GROUP BY
    t.mois
ORDER BY
    total_nuits DESC, chiffre_affaires DESC
LIMIT 3;

```

Mécanique de la requête : Cette requête regroupe les données exclusivement par mois, fusionnant ainsi les récurrences saisonnières abstraction faite de l'année. Le tri lexicographique s'effectue en deux passes : le moteur classe prioritairement par fréquentation (`total_nuits`), et utilise le volume financier (`chiffre_affaires`) comme critère de départage strict en cas d'égalité sur le nombre de nuits.



```

hotel_dw=# SELECT
    t.mois,
    SUM(f.nombre_nuits) AS total_nuits,
    SUM(f.montant_paieement) AS chiffre_affaires
FROM
    fact_reservation f
JOIN
    dim_temps t ON f.id_temps = t.id_temps
GROUP BY
    t.mois
ORDER BY
    total_nuits DESC, chiffre_affaires DESC
LIMIT 3;
mois | total_nuits | chiffre_affaires
-----+-----+-----
 8 |         7 |    1020,00
12 |         7 |     960,00
 1 |         5 |     450,00
(3 rows)

```

FIGURE 8 – Résultat : Top 3 des mois à plus forte activité globale

PARTIE VI - Analyse Stratégique

Cette dernière section* synthétise les résultats obtenus via le Data Warehouse pour fournir à la direction des indicateurs clairs sur les performances de la chaîne hôtelière et des axes d'amélioration.

18. Identification des indicateurs de performance clés

Sur la base de l'échantillon de données analysé, les tendances suivantes se dégagent clairement :

- **L'hôtel le plus rentable :** L'établissement **Ocean View** se positionne en tête avec un chiffre d'affaires de 1 020,00, devançant le Riad Al-Andalus (750,00).
- **Le canal le plus performant :** La plateforme externe **Booking.com** domine largement l'acquisition client en générant un chiffre d'affaires de 1 770,00.
- **Les périodes creuses :** L'analyse chronologique révèle une forte baisse d'activité au mois de **Février** (Mois 2), qui ne génère que 90,00 de revenus, en fort contraste avec les pics d'Août (1 020,00) et Décembre (960,00).

- **Les types de chambres à forte demande** : La catégorie **Deluxe Vue Mer** est la plus rentable de la chaîne, confirmant une forte appétence de la clientèle pour les prestations supérieures avec vue.

19. Recommandations stratégiques

Au vu de ces indicateurs, voici trois recommandations stratégiques justifiées pour optimiser la rentabilité de la chaîne :

1. Réduire la dépendance aux OTA (Online Travel Agencies) :

Constat : Booking.com et Expedia génèrent la majorité des revenus, tandis que le Site Web Direct est sous-performant (210,00).

Action stratégique : Lancer des campagnes de fidélisation et garantir le "meilleur tarif" exclusivement sur le site web de la chaîne. La direction doit investir dans l'expérience utilisateur (UX) de son application mobile et de son site pour récupérer cette clientèle et économiser les fortes commissions prélevées par les plateformes externes.

2. Capitaliser sur le balnéaire haut de gamme :

Constat : L'hôtel "Ocean View" et la chambre "Deluxe Vue Mer" sont les produits phares du portefeuille.

Action stratégique : Allouer un budget de rénovation pour convertir certaines chambres standards en chambres "Deluxe" dans les hôtels côtiers (comme le Marina Bay Resort qui sous-performe actuellement à 450,00). Le Yield Management doit également être agressif sur ces chambres très demandées en haute saison pour maximiser la marge.

3. Dynamiser les périodes creuses (Saisonnalité) :

Constat : Le mois de Février subit une chute drastique du taux d'occupation et des revenus.

Action stratégique : Développer des offres B2B ciblant la clientèle "Corporate" (séminaires, team building) durant l'hiver. Parallèlement, concevoir des packages promotionnels (ex : "Week-end Spa & Détente") pour attirer une clientèle locale sur de courts séjours afin d'amortir les coûts fixes de fonctionnement des hôtels durant ces périodes de faible affluence.

Conclusion Générale

Ce projet de Business Intelligence a permis de résoudre les limites analytiques du système transactionnel initial de la chaîne hôtelière. En concevant un Data Warehouse basé sur un modèle dimensionnel en étoile, les données brutes ont été restructurées autour des axes d'analyse stratégiques exigés par la direction : la temporalité, la géographie, l'offre hôtelière et les vecteurs d'acquisition.

L'implémentation physique sous PostgreSQL, validée par l'injection d'un jeu de données représentatif, a confirmé la robustesse de l'architecture. L'exécution des requêtes décisionnelles a prouvé la capacité du système à isoler et calculer instantanément des indicateurs de performance clés (KPI), tels que le taux d'occupation, la rentabilité par catégorie de chambre ou l'efficacité financière des plateformes de réservation.

En définitive, cet outil décisionnel dote la direction d'un socle technique fiable pour piloter ses opérations fondées sur la donnée (Data-Driven). Il rend techniquement possible l'application de stratégies avancées, telles que l'optimisation tarifaire dynamique (Yield Management) en haute saison ou la rationalisation des coûts de distribution marketing. Ce Data Warehouse est intrinsèquement évolutif ; il pourra ultérieurement intégrer de nouvelles dimensions, telles que les données démographiques fines des clients ou les charges opérationnelles, pour analyser la marge nette de la chaîne avec encore plus de précision.